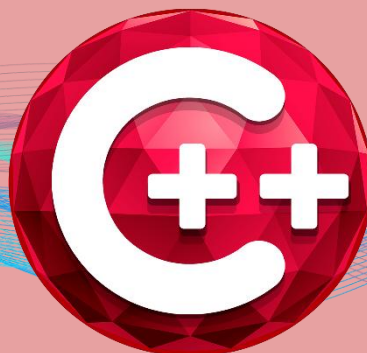


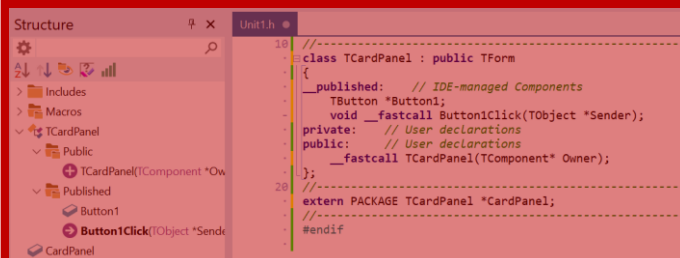


RAD Studio | C++Builder 13

FLORENCE



MÓDULO 1 - BÁSICO



A LINGUAGEM

C++

C++ BUILDER



Criado por: Thiago Santos

MVP Embarcadero. 2026

SUMÁRIO

Sumário	2
Introdução ao Curso de C++Builder 13 Florence	3
O que você vai aprender neste curso	4
Sobre o instrutor.....	5
A linguagem C++	7
Noções básicas da linguagem.....	7
Visão global.....	7
Elementos sintáticos fundamentais	7
Declarações.....	8
Declarações estruturadas	8
Controladores de Loops	13
Tipos de dados.....	18
Tipo estruturado	25
Variáveis	30
Constantes.....	31
Expressões.....	32
Procedimentos e funções.....	34
Procedimentos	34
Funções.....	36
Parâmetros	38

Seja bem-vindo!!

Introdução ao Curso de C++Builder 13 Florence

Este curso foi desenvolvido para apresentar o **C++Builder 13 Florence**, a versão mais recente da plataforma de desenvolvimento da Embarcadero, com foco em quem deseja aprender **C++ de forma prática, produtiva e aplicada ao desenvolvimento de aplicações reais**.

Ao longo desta formação, o aluno será introduzido aos principais conceitos do **C++ funcional aplicado ao C++Builder**, entendendo não apenas a linguagem, mas também como utilizá-la de forma eficiente dentro de um ambiente RAD (Rapid Application Development), que acelera o processo de criação de software sem *abrir mão de desempenho e controle*.

A importância de utilizar o C++Builder 13 Florence

Utilizar a versão mais recente do C++Builder é fundamental para garantir **compatibilidade com tecnologias atuais**, maior estabilidade da IDE e acesso aos recursos mais modernos da linguagem C++. O C++Builder 13 Florence traz melhorias significativas no compilador baseado em Clang, suporte aprimorado ao padrão moderno do C++, além de uma IDE mais rápida, segura e alinhada às necessidades do mercado atual.

Aprender com uma versão atual evita retrabalho futuro, reduz problemas de compatibilidade e prepara o aluno para atuar em projetos profissionais, corporativos e comerciais, utilizando ferramentas reconhecidas e amplamente adotadas por empresas.

O que você vai aprender neste curso

Este curso tem como objetivo fornecer uma **base funcional e prática**, permitindo que o aluno desenvolva aplicações reais desde os primeiros módulos. Entre os principais conhecimentos adquiridos, destacam-se:

- Fundamentos do C++ aplicados ao C++Builder
- Estrutura da IDE e fluxo de desenvolvimento
- Criação de aplicações visuais utilizando VCL
- Manipulação de eventos e componentes
- Lógica de programação aplicada a projetos reais
- Integração básica com banco de dados
- Boas práticas para desenvolvimento em C++Builder

Ao final do curso, o aluno terá conhecimento suficiente para **criar aplicações funcionais**, entender projetos existentes e evoluir para níveis mais avançados da linguagem e da ferramenta.

Por que escolher o C++Builder

O C++Builder se diferencia por unir o **alto desempenho do C++** com a **produtividade do desenvolvimento visual**. Ele permite criar aplicações robustas, rápidas e profissionais com muito menos código do que abordagens tradicionais, mantendo total controle sobre a linguagem e o desempenho.

Além disso, o C++Builder é amplamente utilizado em sistemas corporativos, aplicações industriais, softwares comerciais e soluções que exigem **performance, estabilidade e longevidade**. Aprender C++Builder é investir em uma tecnologia madura, sólida e ainda extremamente relevante no mercado.

Este curso é o primeiro passo para quem deseja dominar o **C++ moderno aplicado ao desenvolvimento profissional**, utilizando uma das ferramentas mais completas e produtivas disponíveis atualmente.

Está pronto? Vamos começar!!

Sobre o instrutor

Meu nome é **Thiago Santos** e sou profissional da área de tecnologia, atuando há vários anos com **desenvolvimento de software, análise de negócios e treinamentos técnicos**, com foco na linguagem **C++** e na plataforma **C++Builder**.

Ao longo da minha trajetória profissional, tive a oportunidade de trabalhar diretamente com projetos reais, ambientes corporativos e soluções que exigem **alto desempenho, estabilidade e manutenção a longo prazo**. Essa experiência prática me permitiu entender não apenas a linguagem C++, mas também como aplicá-la de forma eficiente no dia a dia de empresas e equipes de desenvolvimento.

Sou **MVP da Embarcadero na linguagem C++**, reconhecimento concedido a profissionais que contribuem ativamente para a comunidade, compartilham conhecimento e utilizam as tecnologias Embarcadero em projetos reais. Além disso, atuo como **instrutor, agente de negócios e consultor técnico**, auxiliando empresas e desenvolvedores na adoção e evolução do C++Builder em seus sistemas.

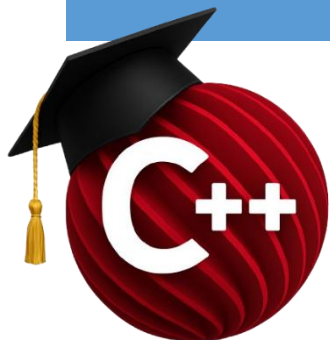
Neste curso, meu objetivo não é apenas ensinar comandos ou conceitos isolados, mas **transmitir a forma correta de pensar, estruturar e desenvolver aplicações em C++Builder**, com base em experiências reais de mercado. Todo o conteúdo foi planejado para ser direto, prático e aplicável, evitando excesso de teoria desconectada da prática.

Acredito que aprender programação vai muito além de escrever código: envolve entender processos, tomar decisões técnicas corretas e criar soluções que realmente funcionem. É exatamente essa visão que compartilho ao longo deste curso.

Seja bem-vindo(a) à formação. Espero que este conteúdo contribua de forma significativa para sua evolução profissional e técnica no desenvolvimento com **C++Builder**.

Meus contatos e plataformas estão disponíveis:

<https://sam-cpp.wordpress.com/>



A LINGUAGEM C++

PERFIL

Este treinamento acadêmico visa apresentar ao aluno um ambiente integrado de desenvolvimento de alta produtividade, que permita a criação de aplicações de todos os tipos e para as mais importantes plataformas da atualidade.

Neste capítulo serão apresentados os conceitos e componentes envolvidos no acesso a dados e componentes de manipulação dos mesmos.

Caso queira se aprofundar e se especializar no conhecimento da ferramenta, sua linguagem, tecnologias e recursos, uma lista de centros de treinamentos oficiais está disponível:

www.embarcadero.com.br

CONTATO

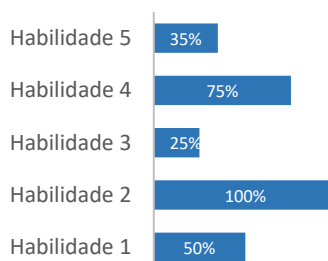
academico@embarcadero.com.br

O que você verá:

Este módulo do treinamento acadêmico é composto por:

- Estrutura de uma Unit
- Declaração de Variáveis
- Estruturas condicionais
- Estruturas de repetição
- Funções e Procedimentos
- Introdução a Classes e Objetos

HABILIDADES



A linguagem C++

NOÇÕES BÁSICAS DA LINGUAGEM

VISÃO GLOBAL

C++ é uma linguagem de alto nível, compilada e fortemente tipada, que suporta tanto o desenvolvimento estruturado quanto o orientado a objetos. Baseada na linguagem C, seus benefícios incluem alto desempenho, grande flexibilidade, suporte a múltiplos paradigmas e ampla compatibilidade com diferentes plataformas. No ambiente do C++Builder, o desenvolvimento visual é integrado por meio de formulários e componentes, permitindo uma programação modular. O C++Builder oferece recursos especiais que suportam o ambiente RAD (Rapid Application Development).

O CONJUNTO DE CARACTERES DO C++

A linguagem C++ utiliza conjuntos de caracteres compatíveis com Unicode, permitindo o uso de caracteres alfabéticos, alfanuméricos e sublinhados em identificadores, dependendo da implementação e do compilador. **Diferente de Delphi, C++ é case-sensitive, ou seja, “a” e “A” são considerados caracteres distintos.**

Os caracteres de espaço e os caracteres de controle ASCII (ASCII 0 a 31, incluindo ASCII 13) são tratados como espaços em branco pelo compilador.

ELEMENTOS SINTÁTICOS FUNDAMENTAIS

Os elementos sintáticos fundamentais, chamado *tokens*, combinam-se para formar expressões, declarações e instruções. A *instrução* descreve uma ação de algoritmos que podem ser executados dentro de um programa. Uma expressão é uma unidade sintática que ocorre dentro de uma instrução e denota um valor. A declaração define um identificador (como o nome de uma função ou variável) que pode ser usado em expressões, declarações e quando necessário, aloca memória para o identificador.

Identificadores

Identificadores em C++ denotam constantes, variáveis, campos, tipos, funções, classes, namespaces e outros elementos do programa. Um identificador pode ter qualquer comprimento, embora alguns compiladores possam impor limites internos. Ele deve começar com uma letra (a–z, A–Z) ou um sublinhado (`_`), e não pode conter espaços. Após o primeiro caractere, são permitidos caracteres alfabéticos, dígitos e underscores. Palavras reservadas da linguagem não podem ser usadas como identificadores. Diferente de Delphi, **C++ é case-sensitive**, portanto, identificadores como *CalculateValue*, *calculateValue*, *calculatevalue* e *CALCULATEVALUE* são considerados **nomes diferentes** pelo compilador.

```
CalculateValue
calculateValue
calculatevalue
CALCULATEVALUE
```

DECLARAÇÕES

Declarações em C++ definem as ações e o comportamento executável de um programa. Declarações simples, como atribuições, chamadas de função e expressões, podem ser combinadas para formar estruturas de controle, como laços, instruções condicionais e outros blocos estruturados que compõem a lógica do programa.

Instruções simples

Uma instrução simples em C++ é aquela que não contém outras declarações internas. Instruções simples incluem atribuições e chamadas de funções ou métodos.

Uma atribuição em C++ tem a forma:

```
variavel = expressão;
```

... Onde **variável** é qualquer identificador válido, incluindo variáveis comuns, ponteiros, membros de classe ou referências, e **expressão** representa qualquer valor ou operação compatível com o tipo da variável. O símbolo = é chamado de **operador de atribuição**.

Uma instrução de atribuição substitui o valor atual da variável pelo resultado da expressão. Por exemplo:

```
i = 3;
```

...atribui o valor 3 à variável i.

A variável à esquerda da atribuição também pode aparecer no lado direito da expressão. Por exemplo:

```
i = i + 1;
```

... incrementa o valor de i.

Assim como em Delphi, cada instrução em C++ termina com um ponto e vírgula (;), que indica o fim da instrução. Dessa forma, uma declaração pode ocupar várias linhas, desde que finalize com ; .

DECLARAÇÕES ESTRUTURADAS

Em C++, declarações estruturadas são formadas a partir de outras declarações simples. Elas são usadas para executar instruções de forma sequencial, condicional ou repetida, permitindo organizar a lógica do programa.

Blocos compostos

Um bloco composto em C++ é uma sequência de instruções simples agrupadas entre chaves {}. As instruções dentro do bloco são executadas na ordem em que aparecem.


```
{
  instruções;
}
```

Exemplo em C++:

```
{
  Z=X;
  X=Y;
  X=Y;
}
```

Em C++, um bloco composto pode ser usado em qualquer lugar onde uma instrução simples seria permitida, permitindo agrupar diversas instruções como se fossem uma única.

Instrução if (Se)

Em C++, a instrução if é usada para executar um comando ou bloco de comandos com base em uma condição lógica. Você pode usar uma das duas decisão condicionais:

- **if** simples
- **if** com **else**

A sintaxe de uma Instrução *if...then* é:

```
if (expressão)
  instrução;
```

Exemplo:

```
if (x > 10)
  y = 0;
```

A expressão usada em uma instrução if deve resultar em um valor booleano. Se essa expressão for verdadeira, a instrução associada ao *if* é executada; se for falsa, nada acontece. Por exemplo:

```
if (j != 0)
  result = i / j;
```

A forma completa da instrução condicional inclui o else:

```
if (expressão)
  instrução1;
else
  instrução2;
```

Novamente, a expressão deve produzir um valor booleano. Se a expressão for verdadeira, **instrução1** será executada. Caso contrário, **instrução2** será executada. Exemplo:

```
if (j == 0)
    return;
else
    result = i / j;
```

Observe que **não deve haver ponto e vírgula após o if (expressão)**, pois isso encerraria a instrução antes do else. Um ponto e vírgula imediatamente depois do if gera um erro lógico — o else perderia sua associação.

Tanto **instrução1** quanto **instrução2** podem ser blocos compostos. Exemplo:

```
if (j == 0)
{
    std::cout << "Divisão inválida.\n";
    return;
}
else
{
    result = i / j;
    std::cout << "Resultado calculado.\n";
}
```

Expressões if podem ser tão simples ou tão complexas quanto necessário. Também é possível aninhar instruções if, criando decisões encadeadas. Por exemplo:

```
if (j != 0)
{
    result = i / j;
    count = count + 1;
}
else if (count == last)
{
    done = true;
}
```

Como algumas instruções if podem ou não incluir um else, o aninhamento pode gerar **ambiguidades**.

Considere o seguinte exemplo:

```
if (expressao1)
    if (expressao2)
        instrucao1;
    else
        instrucao2;
```

Em C++, a associação do else depende diretamente do uso de chaves {}. As duas formas abaixo ilustram como o agrupamento altera o comportamento da instrução condicional.

Sem agrupar o segundo if com chaves:

```
if (expressao1)
    if (expressao2)
        instrucao1;
    else
        instrucao2;
```

... Aqui, o else pertence ao **if mais interno** (o segundo if).

```
if (expressao1)
{
    if (expressao2)
        instrucao1;
}
else
{
    instrucao2;
}
```

Agora, o else está associado ao **primeiro if**, mudando completamente o fluxo da lógica.

No caso de um if aninhado sem o uso de chaves {}, o compilador sempre interpreta a estrutura seguindo a associação padrão do else: **o else pertence ao if mais próximo.**

Assim, a instrução:

```
if (expressao1)
    if (expressao2)
        instrucao1;
    else
        instrucao2;
```

...é sempre analisada como:

```
if (expressao1)
{
    if (expressao2)
        instrucao1;
    else
        instrucao2;
}
```

Existe uma regra em C++ que determina que o else sempre pertence ao if mais próximo. Portanto, quando for necessário que o else esteja associado ao if externo, é obrigatório usar chaves {} para agrupar o bloco interno.

O exemplo correto fica assim:

```
if (expressao1)
{
    if (expressao2)
        instrucao1;
}
else
{
    instrucao2;
}
```

As chaves garantem que o compilador associe o else ao if externo, preservando a lógica desejada.

Instrução switch (Caso)

Quando existem muitas opções possíveis para uma mesma variável ou expressão, usar vários if encadeados pode se tornar pouco prático. Nesses casos, a estrutura switch do C++ é normalmente a solução mais adequada.

A instrução switch tem a seguinte forma geral:

```
switch (expressao)
{
    case valor1:
        instrucao1;
        break;

    case valor2:
        instrucao2;
        break;

    case valor3:
        instrucao3;
        break;

    default:
        instrucao_padrao;
        break;
}
```

A expressão utilizada em um switch deve ser de um tipo integral (como int, char, enum ou tipos equivalentes). Cada rótulo (case) deve representar um valor único — valores repetidos não são permitidos.

O valor da expressão é comparado com cada caso definido na instrução switch. Quando um valor correspondente é encontrado, o bloco associado é executado. Por exemplo, o equivalente em C++ para o trecho apresentado seria:

```

switch (i)
{
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        caption = "Low";
        break;

    case 6:
    case 7:
    case 8:
    case 9:
        caption = "High";
        break;

    case 0:
        caption = "Out of range";
        break;

    // Como C++ não suporta diretamente intervalos em 'case',
    // intervalos como 10..99 devem ser representados manualmente:
    default:
        if (i >= 10 && i <= 99)
            caption = "Out of range";
        else
            caption = "";
        break;
}

```

Em C++, você também pode usar um default, que funciona como o “else” da estrutura. Se nenhum case combinar com o valor da expressão, o bloco default será executado.

CONTROLADORES DE LOOPS

Estruturas de repetição permitem executar um conjunto de instruções várias vezes, utilizando uma condição ou uma variável de controle para determinar quando a repetição deve parar.

Em C++, existem três estruturas principais de loop: while, do-while e for.

Instrução while (Enquanto)

O loop while executa repetidamente um bloco de comandos **enquanto** a condição permanecer verdadeira.

A sintaxe é:

```

while (expressao)
    instrucao;

```

Ou, de forma mais comum, usando um bloco. Por exemplo:

```
while (expressao)
{
    instrucoes;
}
```

A expressão utilizada no while retorna um valor booleano e é avaliada **antes** de cada iteração do loop.

Exemplo equivalente em C++:

```
while (i > 0)
{
    if (i % 2 != 0)    // Odd(i)
        z = z * x;

    i = i / 2;        // div 2
    x = x * x;        // Sqr(x)
}
```

Explicações das conversões:

- Odd(i) foi substituído por $i \% 2 \neq 0$, que verifica se o número é ímpar.
- $i \text{ div } 2$ corresponde a $i / 2$ em C++.
- Sqr(X) corresponde a $x * x$.

Instrução for (Para)

O loop for é usado quando você sabe exatamente quantas vezes deseja repetir um conjunto de instruções. A estrutura define uma variável de controle, um valor inicial, uma condição de parada e uma atualização automática do contador.

Em C++, a sintaxe geral é:

```
for (contador = valorInicial; contador <= valorFinal; contador++)
    instrucao;
```

Para percorrer no sentido decrescente, utiliza-se:

```
for (contador = valorInicial; contador >= valorFinal; contador--)
    instrucao;
```

A instrução **for** inicializa o contador com o valor inicial e, em seguida, executa o bloco repetidamente.

Após cada iteração, o contador é incrementado ou decrementado automaticamente, dependendo da forma do loop.

No formato crescente (to), o contador é incrementado. No formato decrescente (downto), o contador é decrementado.

O loop continua enquanto o contador estiver dentro da faixa definida, e a última execução ocorre quando o contador atinge exatamente o valor final. Ou seja, a instrução é executada uma vez para cada valor dentro do intervalo. O contador **não deve ser alterado manualmente** dentro do loop.

Exemplo equivalente em C++:

```
for (int i = 10; i <= 100; i++)
{
    std::cout << "looping\n";
}
```

Loop for baseado em intervalo (range-based for)

C++ possui uma variação do comando for destinada a percorrer elementos de containers e coleções de forma simples. Essa estrutura é chamada de **range-based for**, e é o equivalente direto ao for...in...do.

A sintaxe é:

```
for (auto elemento : expressao)
    instrução;
```

Onde:

- **expressao** é um container, como std::vector, std::array, std::string, entre outros;
- **elemento** recebe cada item do container, automaticamente, a cada iteração.

Exemplo:

```
std::vector<int> numeros = {1, 2, 3, 4};

for (auto n : numeros)
{
    std::cout << n << "\n";
}
```

Esse formato torna a iteração sobre listas, vetores e coleções muito mais simples e direta.

```
DynamicArray<String> strs;

// ... preencher o array ...

for (const String &item : strs)
{
    ShowMessage(item);
}
```

DynamicArray<String>

Tipo de vetor dinâmico usado no C++Builder para armazenar strings.

for (const String &item : strs)

O *range-based for* do C++11 percorre cada elemento da coleção:

- item é atualizado a cada iteração
- const impede que seja modificado — exatamente como na descrição
- funciona para:
 - vetores dinâmicos (DynamicArray)
 - cadeias (String)
 - std::vector
 - coleções FMX/VCL com iteradores

ShowMessage(item)

Executa algo para cada elemento, equivalente ao exemplo.

```
DynamicArray<String> str;
str.set_length(3);
str[0] = "Primeiro";
str[1] = "Segundo";
str[2] = "Terceiro";

for (const String &item : str)
{
    ShowMessage(item);
}
```

Instrução repeat...until (Repita)

Em C++, o equivalente ao comportamento "executa pelo menos uma vez e testa no final" é o comando:

```
do { ... } while (condição);
```

Mas observe:

- repeat ... until condição termina quando **condição é verdadeira**
- do ... while (condição) continua enquanto **condição é verdadeira**

Portanto, precisamos inverter a lógica.

Sintaxe convertida**repeat****instruções****until expressão;**

Converte para:

do {**instruções;****} while (!(expressão));**

Exemplo convertido

Código original:

```
repeat
K := I mod J;
I := J;
J := K;
until J = 0;
```

Código equivalente em C++:

```
do
{
K = I % J;
I = J;
J = K;
}
while (J != 0);
```

Explicação breve

- O bloco do { ... } while (...) sempre executa pelo menos uma vez — exatamente o que queremos.
- A condição é repetida enquanto for verdadeira.
- Como o original pára quando J = 0, em C++ repetimos enquanto J ≠ 0.

break e continue

Os procedimentos padrão *break* e *continue* são usados para sair do loop imediatamente, sem ter uma condição particular.

Break é usado para sair do loop e caso haja alguma instrução após o mesmo, ela será executada. Por exemplo:

```
while (true)
{
    std::string i;
    std::getline(inputFile, i);

    if (std::stoi(i) == 5)
        break;

    ShowMessage(i.c_str());
}
```

Neste exemplo, o loop é executado indefinidamente até que alguma condição o interrompa. O loop continua lendo valores de um arquivo e exibindo-os em uma caixa de mensagem, até que o valor '5' seja encontrado. Quando o comando break é executado, o fluxo do programa salta para a primeira instrução após o bloco do loop.

O comando continue funciona de forma semelhante, porém, em vez de sair do loop, ele interrompe apenas a iteração atual e inicia imediatamente a próxima. Por exemplo:

```
while (i < 10)
{
    std::string linha;
    std::getline(inputFile, linha);
    i = std::stoi(linha);

    if (i < 5)
        continue;

    ShowMessage("This number will be greater than or equal to 5: " + std::to_string(i).c_str());
}
```

TIPOS DE DADOS

Em C++ Builder, um **tipo** define o formato e o conjunto de valores que uma variável pode armazenar, além das operações possíveis sobre ela. Toda variável deve declarar seu tipo, e esse tipo determina:

- o tamanho em memória,
- o intervalo de valores possíveis,
- o comportamento em operações matemáticas e lógicas,
- e a forma como funções recebem ou retornam valores.

C++ é uma linguagem **fortemente tipada**, permitindo que o compilador detecte erros de tipo antes da execução. Em situações onde é necessário converter tipos, C++ oferece mecanismos seguros como *casts*, além de suportar conversões automáticas em contextos válidos.

Tipo Integer (inteiro)

A tabela a seguir mostra os tipos inteiros mais usados no **C++ Builder** e suas faixas típicas:

Tipo C++ Builder	Faixa de valores	Tamanho	Assinatura
int	-2,147,483,648 ... 2,147,483,647	32 bits	signed
unsigned int	0 ... 4,294,967,295	32 bits	unsigned
signed char	-128 ... 127	8 bits	signed
short	-32,768 ... 32,767	16 bits	signed
long	-2,147,483,648 ... 2,147,483,647	32 bits	signed
long long	$-2^{63} \dots 2^{63}-1$	64 bits	signed
unsigned char (byte)	0 ... 255	8 bits	unsigned
unsigned short (word)	0 ... 65,535	16 bits	unsigned
unsigned long	0 ... 4,294,967,295	32 bits	unsigned

Observação sobre C++ Builder

No C++ Builder, existem também tipos equivalentes usados frequentemente no ambiente RAD:

- `__int64` → inteiro de 64 bits
- `DWORD` → unsigned 32 bits
- `WORD` → unsigned 16 bits
- `BYTE` → unsigned 8 bits

Esses tipos são comuns em APIs do Windows.

Exemplo de declaração em C++ Builder

```
int contador = 0;
unsigned int quantidade = 1500;
short nivel = -12;
long long valorGrande = 900000000000;
BYTE code = 255;
WORD tamanho = 1024;
```

Em C++, os números inteiros também são categorizados conforme a quantidade de memória que ocupam e o intervalo de valores que podem representar. Os tipos inteiros “genéricos”, como `int` e `unsigned int`, são recomendados na maioria dos casos, pois oferecem melhor desempenho e são otimizados para a arquitetura da CPU e do sistema operacional. Esses tipos normalmente correspondem ao tamanho inteiro nativo da plataforma, proporcionando operações matemáticas mais rápidas e maior eficiência em nível de hardware.

Tipo Real (real)

Em C++ Builder, números reais são representados através de tipos de **ponto flutuante**, usados para armazenar valores com casas decimais. Esses tipos seguem os padrões de hardware da plataforma Win32 e sua precisão varia conforme o tamanho em bytes.

A tabela abaixo mostra os principais tipos numéricos reais usados em C++ Builder:

Tipo C++	Faixa aproximada	Precisão (dígitos significativos)	Tamanho
float	$\sim \pm 1.5 \times 10^{-45}$ a $\pm 3.4 \times 10^{38}$	6–7 dígitos	4 bytes
double	$\sim \pm 5.0 \times 10^{-324}$ a $\pm 1.7 \times 10^{308}$	15–16 dígitos	8 bytes
long double (C++ Builder 32-bit)	$\sim \pm 3.6 \times 10^{4951}$ a $\pm 1.1 \times 10^{4932}$	18–20 dígitos	10 bytes
__int64 (Comp equivalente)	não é ponto flutuante — inteiro de 64 bits	—	8 bytes
Currency (C++ Builder)	valor fixo com 4 casas decimais	19–20 dígitos	8 bytes

Observações importantes no C++ Builder

✓ **long double**: No C++ Builder Win32, `long double` mantém o formato Extended 80-bit (10 bytes), equivalente ao tipo *Extended*.

✓ **Currency**: Em C++ Builder, o tipo `Currency` existe como:

- `Currency` valor;

Ele usa representação fixa com 4 casas decimais, ideal para cálculos financeiros.

- `Real48`

O tipo Real48 não existe em C++ moderno, e não é utilizado em C++ Builder. Exemplo:

```
float a = 3.14f;  
double b = 2.718281828;  
long double c = 1.234567890123456789L;
```

```
Currency preco;  
preco = 199.99;
```

Tipo Char (caractere)

O tipo char em C++Builder tem o mesmo conceito. Ele armazena um único caractere de 8 bits, que tipicamente representa um caractere ASCII. Em C++Builder, o char é usado para representar dados de texto que seguem o conjunto de caracteres ASCII.

- **char (caractere):** Em C++Builder, o tipo char armazena um único caractere de 8 bits, normalmente um caractere ASCII. Ele é semelhante ao AnsiChar de Delphi, que também usa o conjunto de caracteres de 8 bits.
- **wchar_t (WideChar):** Este tipo em C++Builder é usado para armazenar caracteres de 16 bits, o que corresponde ao WideChar de Delphi. Ele é utilizado para representar caracteres Unicode. O tipo wchar_t é usado para suportar caracteres fora do conjunto ASCII, como caracteres de línguas não latinas e outros caracteres especiais.
- **char16_t e char32_t:** Em C++11, o padrão introduziu tipos char16_t e char32_t para representar caracteres Unicode de 16 e 32 bits, respectivamente. Estes tipos também podem ser usados em C++Builder, mas o uso de wchar_t é mais comum para compatibilidade com o padrão C++ anterior.

Tipo String (palavra)

Em C++Builder, strings modernas não são mais tratadas como simples arrays fixos de caracteres. As implementações antigas baseadas em arrays de 8 bits apresentavam diversas limitações, como tamanho máximo reduzido e ausência de terminação com caractere binário zero (NULL). Essas limitações dificultavam a interoperabilidade com sistemas e APIs que utilizam strings terminadas em NULL, comuns em ambientes Windows, UNIX e em linguagens como C e Assembly.

Para resolver esses problemas, o C++Builder utiliza tipos de string mais avançados, que oferecem gerenciamento automático de memória, suporte a Unicode e compatibilidade com APIs do sistema operacional.

Existem basicamente dois conceitos de string no C++Builder:

- **Strings curtas (short strings):** Baseadas em buffers de tamanho limitado, semelhantes às strings clássicas, com restrições de comprimento e uso mais específico.
- **Strings longas/dinâmicas:** São alocadas dinamicamente, gerenciadas automaticamente pelo compilador e pela RTL, podendo crescer conforme a

memória disponível. Essas strings são sempre terminadas em NULL, facilitando a integração com APIs externas.

As strings dinâmicas possuem um **contador de referência interno**, invisível ao programador. Esse contador permite que múltiplas variáveis compartilhem a mesma instância de string de forma eficiente. Quando o contador de referência chega a zero, a memória associada é liberada automaticamente.

Strings são delimitadas por aspas simples. Para incluir uma aspa simples dentro do próprio texto da string, utiliza-se a duplicação do caractere, que será interpretada como uma única aspa incorporada.

O C++Builder fornece um amplo conjunto de rotinas internas para manipulação de strings, incluindo concatenação, comparação, cópia, busca e conversão. Além disso, as strings podem ser acessadas como se fossem arrays de caracteres, permitindo a leitura ou modificação direta de posições específicas dentro da string.

```
String s;
char ch1, ch2;

s = "Hello";
ch1 = s[1];
ch2 = s[2];

ShowMessage("Characters 1 and 2 are " + String(ch1) + " " + String(ch2));
```

A tabela a seguir demonstra todos os tipos de strings suportados no **C++Builder**, suas características e usos:

Tipo	Tamanho máximo	Memória	Uso
ShortString	255 caracteres	2 a 256 bytes	Compatibilidade com código legado
AnsiString	$\sim 2^{31}$ caracteres	8 bits por caractere	Strings ANSI, DBCS, MBCS e compatibilidade com APIs antigas
UnicodeString (String)	$\sim 2^{30}$ caracteres	4 bytes a 2 GB	Strings Unicode, aplicações multilíngues e servidores multiusuários
WideString	$\sim 2^{30}$ caracteres	4 bytes a 2 GB	Strings Unicode usadas principalmente em aplicações COM

No C++Builder moderno, o tipo `String` é, por padrão, um `UnicodeString`, oferecendo suporte nativo a Unicode, gerenciamento automático de memória e contagem de referência.

O tipo `WideString` ainda é utilizado em cenários específicos, principalmente para interoperabilidade com COM, enquanto `AnsiString` e `ShortString` são mantidos principalmente por motivos de compatibilidade com código legado.

Tipo Boolean (verdadeiro ou falso)

Em C++, o tipo **booleano** é representado pelo tipo **bool**. O **bool** armazena um valor lógico, que pode ser **true** (verdadeiro) ou **false** (falso).

Embora o C++ não tenha exatamente os mesmos tipos de `Bytebool`, `Wordbool` ou `Longbool` como em outras linguagens, ele oferece a flexibilidade de representar valores booleanos diretamente através do tipo **bool**.

Características do tipo bool em C++:

- Os valores **true** e **false** são representações internas de 1 e 0, respectivamente.
- O tipo **bool** é amplamente utilizado em C++ para controle de fluxo, expressões condicionais e outras operações lógicas.
- Em C++, não é necessário um tipo separado para representar booleans de 8 bits, 16 bits ou 32 bits, já que **bool** já é otimizado para representar valores lógicos de forma eficiente.

Em resumo, o tipo booleano em C++ usa o tipo **bool**, com as constantes **true** e **false** para representar valores lógicos.

Tipo enumerado

Em C++Builder, é possível declarar **tipos enumerados** para definir um conjunto ordenado de valores nomeados. Um tipo enumerado representa um conjunto finito de constantes inteiras, onde cada identificador corresponde a um valor sequencial, atribuído automaticamente a partir de zero, a menos que seja especificado explicitamente.

Os valores de um tipo enumerado não possuem significado próprio além dos nomes que os identificam; eles servem para tornar o código mais legível, seguro e fácil de manter.

A sintaxe básica para declarar um tipo enumerado em C++Builder é:

```
enum Nome { val1, val2, ..., valn };
```

Cada identificador representa um valor distinto dentro do tipo enumerado, e o compilador atribui valores inteiros crescentes conforme a ordem da declaração. Por exemplo, a declaração:

```
enum Suit { Club, Diamond, Heart, Spade };
```

... Esse código define um **tipo enumerado** chamado Suit, cujos valores possíveis são Club, Diamond, Heart e Spade.

Você pode usá-los da seguinte maneira:

```
Suit MySuit;

MySuit = Club;
MySuit = Heart;
MySuit = Spade;
```

Neste código, o tipo enumerado Suit é utilizado para declarar a variável MySuit, que pode assumir os valores Club, Heart ou Spade.

Tipo Subrange (subfaixa)

Em C++Builder, um **tipo subrange** pode ser simulado utilizando **tipos inteiros** e restrições nos valores possíveis para variáveis desse tipo. No entanto, C++ não tem uma estrutura de tipo de subfaixa diretamente, como em algumas outras linguagens. Em vez disso, você pode usar **tipos inteiros** com **valores limitados** ou **constantes** para representar subfaixas.

Uma forma comum de restringir valores a um intervalo específico é utilizando **enumeração** ou **constantes** para garantir que os valores permaneçam dentro de um intervalo

Por exemplo se você declarar um tipo enumerado:

```
enum TColors { Red, Blue, Green, Yellow, Orange, Purple, White, Black };
```

Em C++Builder não existe um tipo **Subrange** nativo, mas é possível **representar a subfaixa** a partir de um tipo enumerado usando o próprio enum e controlando os limites por convenção.

Considerando o enumerado já definido:

```
enum TColors { Red, Blue, Green, Yellow, Orange, Purple, White, Black };
```

A subfaixa pode ser representada conceitualmente assim:

```
typedef TColors TMyColors;
```

Onde os valores válidos de TMyColors ficam **restritos ao intervalo**:

```
Green, Yellow, Orange, Purple, White
```

Ou seja, todos os valores entre Green e White, inclusive.

Na prática, a restrição da subfaixa deve ser **garantida por validações no código**, verificando se o valor atribuído está dentro dos limites desejados, já que o C++Builder não impõe essa restrição automaticamente no nível do tipo.

TIPO ESTRUTURADO

Em C++Builder, tipos estruturados como matrizes e registros podem ser usados da maneira equivalente, mas com algumas diferenças na sintaxe. Aqui está a conversão de como você pode declarar e usar matrizes em C++Builder.

Matrizes estáticas

Em C++Builder, uma **matriz estática** pode ser declarada usando a sintaxe `array[index]` do tipo desejado. A sintaxe básica é a seguinte:

```
// Declaração de uma matriz estática  
char MyArray[100];
```

Neste exemplo, `MyArray` é uma matriz de 100 elementos do tipo `char`. Similar a outras linguagens, o índice da matriz começa em 0, ou seja, `MyArray[0]` é o primeiro elemento e `MyArray[99]` é o último.

Criando Matrizes como Tipo

Você também pode definir um tipo para a matriz, da mesma forma que em Delphi. Para isso, você pode usar a palavra-chave `typedef` para criar um tipo de matriz:

```
// Definindo um tipo para a matriz  
typedef char MyArray[100];  
  
// Declarando variáveis deste tipo  
MyArray FirstArray;  
MyArray SecondArray;
```

Explicação

- Em **C++Builder**, a declaração de uma matriz estática é feita diretamente com o tipo seguido pelo número de elementos entre colchetes, como no exemplo `char MyArray[100]`.
- **Matrizes estáticas** têm um número fixo de elementos que não podem ser alterados em tempo de execução, ao contrário das **matrizes dinâmicas**.
- Ao declarar uma **matriz estática**, se os elementos não forem inicializados, o conteúdo pode conter **valores aleatórios** ou lixo de memória, como mencionado.

Código Final

Aqui está um exemplo completo de como você pode implementar em C++Builder:

```

#include <vcl.h>
#include <iostream>
using namespace std;

// Definindo um tipo para a matriz
typedef char MyArray[100];

int main() {
    // Criando variáveis do tipo MyArray
    MyArray FirstArray;
    MyArray SecondArray;

    // Atribuindo valores
    FirstArray[0] = 'A';
    SecondArray[99] = 'Z';

    // Exibindo os valores atribuídos
    cout << "FirstArray[0]: " << FirstArray[0] << endl;
    cout << "SecondArray[99]: " << SecondArray[99] << endl;

    return 0;
}

```

Neste exemplo:

- **MyArray** é um tipo de matriz de 100 caracteres.
- **FirstArray** e **SecondArray** são variáveis do tipo **MyArray**, que são matrizes de 100 caracteres.
- Atribuímos valores para o primeiro e o último elemento das matrizes e os exibimos.

Matriz dinâmica

Em C++Builder, **matrizes dinâmicas** não possuem tamanho fixo. A memória é alocada e realocada dinamicamente conforme o tamanho definido em tempo de execução. Esse comportamento é obtido por meio de **arrays dinâmicos da RTL** ou utilizando contêineres padrão.

Uma matriz dinâmica é definida apenas pelo tipo base, sem especificar o tamanho no momento da declaração. A alocação de memória ocorre quando o tamanho é definido explicitamente.

Declaração de matriz dinâmica:

```
DynamicArray<double> MyFlexibleArray;
```

Essa declaração define uma matriz dinâmica unidimensional do tipo **double**, mas **nenhuma memória é alocada** neste momento.

Alocação de memória (definição do tamanho):

```
MyFlexibleArray.Length = 20;
```

Isso aloca espaço para **20 valores do tipo double**, indexados de 0 a 19.

Características das matrizes dinâmicas

- O tamanho pode ser alterado em tempo de execução.
- A memória é gerenciada automaticamente.
- Os índices são sempre **inteiros**.
- A indexação **sempre começa em 0**.
- O conteúdo inicial não é garantido, a menos que seja explicitamente inicializado.

Matrizes dinâmicas são ideais quando o tamanho dos dados só é conhecido em tempo de execução ou quando é necessário redimensionar a estrutura ao longo da execução do programa.

Sets (conjunto)

Em C++Builder, um **set (conjunto)** representa uma coleção de valores únicos de um mesmo **tipo ordinal**, sem ordem definida e sem duplicação de elementos, de forma semelhante aos conjuntos matemáticos.

Esse conceito é implementado por meio do template **Set** da RTL, que trabalha normalmente com **tipos enumerados** ou valores inteiros dentro de um intervalo definido.

A declaração genérica de um conjunto em C++Builder é feita da seguinte forma:

```
Set<baseType, minValue, maxValue> NomeDoConjunto;
```

Onde:

- **baseType** é o tipo ordinal (geralmente um enum)
- **minValue** e **maxValue** definem o intervalo válido de valores do conjunto

Cada valor pode estar presente ou ausente no conjunto, nunca duplicado, e a ordem dos elementos não é significativa.

Conjuntos são usados principalmente para representar **grupos de opções, flags e combinações de valores discretos**, com operações eficientes como inclusão, exclusão e verificação de pertencimento.

Por exemplo:

```
enum TDirection { North, East, South, West };

typedef Set<TDirection, North, West> TDirections;

TDirections aVerticalDirs;

aVerticalDirs << East << West;
```

Nesse código:

- **TDirection** define o tipo enumerado.
- **TDirections** define um conjunto baseado nesse enumerado.
- **aVerticalDirs** é uma variável do tipo conjunto.
- Os valores **East** e **West** são inseridos no conjunto.

Records (registro)

Em C++Builder, o **tipo de dados Record** é representado pelo **struct**. Um struct permite agrupar, em uma única declaração, um conjunto lógico de valores que podem ser de **tipos iguais ou diferentes**.

Cada valor contido dentro da estrutura é chamado de **campo** (ou membro), e pode ser acessado individualmente por meio da variável que representa o registro.

Esse tipo de estrutura é amplamente utilizado para representar entidades compostas, como coordenadas, configurações ou qualquer conjunto de dados relacionados.

Por exemplo, um vetor bidimensional pode ser representado por uma estrutura contendo os campos que armazenam as coordenadas **X** e **Y**, mantendo os dados organizados e semanticamente claros.

Em C++Builder, a declaração de um **Record** é feita utilizando a palavra-chave **struct**. A sintaxe equivalente é:

```
struct recordTypeName {
    type1 fieldList1;
    ...
    typen fieldListn;
};
```

Por exemplo, a declaração a seguir cria um tipo de registro chamado TDateRec:

```
enum TMonth { Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };

struct TDateRec {
    int Ano;
    TMonth Mes;
    int Dia; // valores válidos: 1 a 31
};
```

Em C++Builder, a declaração de variáveis para o tipo TDateRec e o acesso aos seus campos é feita de forma semelhante, utilizando o operador de ponto (.) para acessar os campos da instância. A seguir está a conversão do exemplo:

```
enum TMonth { Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };

struct TDateRec {
    int Ano;
    TMonth Mes;
    int Dia;
};
```

Declaração e uso das variáveis:

```
TDateRec Record1, Record2;

Record1.Ano = 1904;
Record1.Mes = Jun;
Record1.Dia = 16;
```

Explicação:

1. **Definindo o tipo:** A estrutura TDateRec contém três campos: Ano (inteiro), Mes (do tipo TMonth, que é um tipo enumerado), e Dia (inteiro, representando o número entre 1 e 31).
2. **Declarando as variáveis:** Record1 e Record2 são instâncias do tipo TDateRec. Elas são criadas e a memória é alocada quando você declara essas variáveis.
3. **Acessando os campos:** Para acessar ou modificar os campos de Record1, você usa a notação Record1.Ano, Record1.Mes, e Record1.Dia, respectivamente.

O uso do operador de ponto (.) permite que você acesse os campos da estrutura como se fossem variáveis, da mesma forma que na linguagem Delphi.

Tipo Variant (variável)

Em C++Builder, o **tipo Variant** é utilizado quando é necessário manipular dados cujo **tipo pode variar** ou **não é conhecido em tempo de compilação**. Uma variável Variant pode assumir diferentes tipos em tempo de execução, oferecendo grande flexibilidade no tratamento de dados dinâmicos.

Essa flexibilidade tem um custo: variáveis do tipo Variant **consomem mais memória** e **suas operações são mais lentas** quando comparadas a tipos fortemente tipados. Além disso, erros decorrentes de operações inválidas com Variant geralmente só são detectados **em tempo de execução**, enquanto erros equivalentes com tipos estáticos são identificados **em tempo de compilação**.

O C++Builder também permite a criação de **tipos Variant personalizados**, possibilitando a extensão do comportamento padrão para necessidades específicas.

Por padrão, um Variant pode armazenar valores de diversos tipos simples, como números, strings, datas e valores lógicos. No entanto, **não pode conter diretamente** tipos estruturados ou de baixo nível, tais como:

- Registros (struct)
- Conjuntos

- Matrizes estáticas
- Classes
- Referências de classe
- Ponteiros

Em resumo, Variant é uma solução poderosa para cenários dinâmicos, mas deve ser utilizada com critério devido ao impacto em desempenho, uso de memória e segurança de outros tipos:

```
Variant V1, V2, V3, V4, V5;
int I;
double D;
String S;

V1 = 1;           // valor inteiro
V2 = 1234.5678;   // valor real
V3 = "Hello world!"; // valor string
V4 = "1000";      // valor string

V5 = V1 + V2 + V4; // valor real: 2235.5678

I = V1;          // I = 1 (inteiro)
D = V2;          // D = 1234.5678 (real)
S = V3;          // S = "Hello world!"
I = V4;          // I = 1000 (inteiro, conversão implícita)
S = V5;          // S = "2235.5678" (string)
```

Esse código demonstra como o tipo Variant em C++Builder pode armazenar valores de diferentes tipos e realizar conversões implícitas em tempo de execução, permitindo operações aritméticas e atribuições entre tipos distintos.

VARIÁVEIS

Em C++Builder, uma **variável** é um identificador cujo valor pode mudar durante a execução do programa. Ela representa um **local na memória**, e o nome da variável é usado para **ler ou gravar** valores nesse endereço.

A declaração de uma variável define o **tipo de dado** que ela pode armazenar, permitindo ao compilador reservar a quantidade adequada de memória e aplicar verificações de tipo.

A sintaxe básica para a declaração de uma variável em C++Builder é:

```
tipo Identificador;
```

Onde:

- **tipo** especifica o tipo de dado da variável
- **Identificador** é o nome da variável

Após declarada, a variável pode ter seu valor alterado livremente durante a execução do programa. Por exemplo:

```
int I;
```

...declara uma variável do tipo inteiro, enquanto:

```
double X, Y;
```

Em C++Builder, **não existe a palavra reservada var**. As variáveis são declaradas diretamente pelo tipo, e declarações consecutivas não exigem nenhuma palavra especial.

A conversão fica assim:

```
double X, Y, Z;
int I, J, K;
int Digit; // valores válidos: 0 a 9
bool Okay;
```

Observações importantes

- O intervalo 0..9 não existe como tipo nativo; por isso Digit é declarado como int, e a restrição deve ser garantida por validação no código.
- O tipo Boolean é representado por bool.
- Variáveis declaradas **dentro de uma função ou método** são **variáveis locais**.
- Variáveis declaradas **fora de funções**, em escopo de unidade ou global, são **variáveis globais**.

O escopo da variável determina onde ela pode ser acessada e por quanto tempo permanece válida durante a execução do programa.

CONSTANTES

Em C++Builder, uma **constante** é um identificador cujo valor **não pode ser alterado** após a definição. Constantes são usadas para representar valores fixos, tornando o código mais claro, seguro e fácil de manter.

A forma mais comum de declarar uma constante em C++Builder é utilizando a palavra-chave **const**.

A conversão do exemplo fica assim:

```
const int MaxValue = 237;
```

Nesse caso, MaxValue é uma constante que sempre representa o valor inteiro 237.

Sintaxe para declaração de constantes:

```
const tipo identificador = expressao_constante;
```

Onde:

- **tipo** define o tipo do valor constante
- **identificador** é o nome da constante
- **expressao_constante** é uma expressão que pode ser avaliada pelo compilador, sem a necessidade de executar o programa

O compilador garante que o valor da constante não seja modificado durante a execução do programa.

EXPRESSÕES

Em C++Builder, uma **expressão** é qualquer construção que **produz um valor**. As expressões mais simples são **variáveis** e **constantes**. Expressões mais complexas são formadas pela combinação desses elementos básicos usando **operadores**, **chamadas de função**, **conversões de tipo**, entre outros recursos da linguagem.

Operadores

Os **operadores** em C++Builder funcionam como operações predefinidas da linguagem e atuam sobre um ou mais **operandos**.

Por exemplo, a expressão:

X+Y

é composta pelas variáveis X e Y (operandos) e pelo operador +. Quando X e Y representam valores numéricos inteiros ou reais, o resultado da expressão é a **soma** desses valores.

Operadores podem ser utilizados para realizar operações aritméticas, lógicas, relacionais e de atribuição, permitindo a construção de expressões simples ou altamente complexas que são avaliadas em tempo de execução e retornam um valor.

Operadores aritméticos

Em C++Builder, os **operadores aritméticos** são usados para realizar cálculos numéricos com valores inteiros e reais. A tabela a seguir mostra os operadores equivalentes e seu comportamento:

Operador	Operação	Tipos de operandos	Tipo de resultado	Exemplo
+	Adição	inteiro, real	inteiro, real	X + Y
-	Subtração	inteiro, real	inteiro, real	Result - 1
*	Multiplicação	inteiro, real	inteiro, real	P * InterestRate
/	Divisão	inteiro, real	real	X / 2
/ + conversão inteira	Divisão inteira	inteiro	inteiro	Total / UnitSize
%	Resto da divisão	inteiro	inteiro	Y % 6

Observações

- O operador / sempre realiza **divisão real** quando usado com tipos de ponto flutuante.
- Para obter **divisão inteira**, ambos os operandos devem ser inteiros.
- O operador % é utilizado para obter o **resto da divisão inteira**, equivalente ao operador mod.
- Não existe um operador específico chamado div; a divisão inteira é obtida naturalmente com tipos inteiros.

Esses operadores permitem a construção de expressões aritméticas simples ou complexas em C++Builder.

Operadores Booleanos

Em C++Builder, os **operadores booleanos** trabalham com operandos do tipo bool e retornam um valor booleano (true ou false). Eles são usados para combinar, inverter ou comparar expressões lógicas.

A equivalência dos operadores é a seguinte:

Operador	Operação	Tipos de operandos	Tipo de resultado	Exemplo
!	Negação (not)	bool	bool	!(CInMySet)
&&	Conjunção (and)	bool	bool	Done && (Total > 0)
			Disjunção (or)	bool
^	Disjunção exclusiva	bool	bool	A ^ B

Observações

- O operador ! inverte o valor lógico da expressão.
- Os operadores && e || possuem **avaliação curta** (*short-circuit*), ou seja, a segunda expressão só é avaliada se necessário.
- O operador ^, quando aplicado a valores booleanos, funciona como **XOR lógico**.
- Todos esses operadores retornam um valor do tipo bool.

Esses operadores permitem a construção de expressões lógicas simples ou complexas em C++Builder.

Operadores relacionais

Em C++Builder, os **operadores relacionais** são usados para comparar dois valores e retornam um resultado do tipo bool.

A tabela a seguir mostra os operadores relacionais e seus equivalentes:

Operador	Operação	Tipos de operandos	Tipo de resultado	Exemplo
==	Igual	tipos simples, classes, interfaces, strings	bool	I == Max
!=	Diferente	tipos simples, classes, interfaces, strings	bool	X != Y
<	Menor que	tipos simples, strings, PChar	bool	X < Y
>	Maior que	tipos simples, strings, PChar	bool	Len > 0
<=	Menor ou igual a	tipos simples, strings, PChar	bool	Cnt <= I
>=	Maior ou igual a	tipos simples, strings, PChar	bool	I >= 1

PROCEDIMENTOS E FUNÇÕES

Procedimentos e funções são representados por **funções C++**. Ambas são blocos independentes de código que podem ser chamados a partir de diferentes pontos do programa.

- Uma **função** é uma rotina que **retorna um valor**.
- Um **procedimento** é uma rotina que **não retorna valor**, sendo representado por uma função com tipo de retorno void.

PROCEDIMENTOS

Procedimentos são definidos como funções cujo tipo de **retorno é void**. Eles podem conter parâmetros, declarações locais e um bloco de instruções que será executado quando a rotina for chamada.

A estrutura geral de um procedimento em C++Builder é:

```
void NomeProcedimento(lista_de_parametros)
{
    // declarações locais
    // instruções
}
```

Onde:

- **NomeProcedimento** é qualquer identificador válido.
- **lista_de_parametros** é opcional e define os dados recebidos pela rotina.
- As **declarações locais** são variáveis visíveis apenas dentro do procedimento.
- O corpo do procedimento contém as instruções executadas quando ele é chamado.

Essa estrutura permite organizar o código em rotinas reutilizáveis, facilitando a manutenção, leitura e modularização do programa.

Por exemplo:

```
void NumString(int N, String &S)
{
    int V = Abs(N);
    S = "";

    do {
        S = String(char((V % 10) + '0')) + S;
        V = V / 10;
    } while (V != 0);

    if (N < 0)
        S = "-" + S;
}
```

Esse procedimento recebe um inteiro N e retorna sua representação textual em S, usando passagem por **referência** (String &S).

Dada a declaração do procedimento, você pode chamá-lo em **C++Builder** da seguinte forma:

```
String MyString;

NumString(17, MyString);
```

Essa chamada atribui o valor "17" à variável MyString, que deve ser do tipo String.

Dentro do corpo de uma função ou procedimento em C++Builder, é possível utilizar:

- Variáveis declaradas localmente dentro da própria função
- Identificadores visíveis no escopo onde a função foi declarada
- Os **parâmetros da lista de parâmetros**, que se comportam como **variáveis locais**

Os nomes dos parâmetros já fazem parte do escopo da função, portanto **não devem ser redeclarados** dentro do corpo da rotina, assim como ocorre com qualquer variável local em C++.

FUNÇÕES

Em C++Builder, uma **função** é semelhante a um procedimento, com a diferença de que **retorna um valor**. A declaração de uma função especifica explicitamente o **tipo de retorno** e utiliza a instrução **return** para devolver o resultado ao chamador.

A forma geral de uma função em C++Builder é:

```
TipoDeRetorno NomeFuncao(lista_de_parametros)
{
    // declarações locais
    // instruções
    return valor;
}
```

Onde:

- **NomeFuncao** é qualquer identificador válido.
- **TipoDeRetorno** define o tipo do valor retornado pela função.
- **lista_de_parametros** é opcional.
- As **declarações locais** são variáveis válidas apenas dentro da função.
- O bloco de instruções contém o código executado quando a função é chamada.

As regras de escopo dentro de funções são as mesmas aplicadas a procedimentos: é possível utilizar variáveis locais, parâmetros e identificadores visíveis no escopo onde a função foi declarada.

Diferente de algumas linguagens, em C++Builder o valor de retorno **não é atribuído ao nome da função** nem a uma variável implícita; o retorno é feito explicitamente por meio da instrução **return**, que define o valor final devolvido pela função.

Por exemplo:

```
int WF()
{
    return 17;
}
```

As duas formas apresentadas produzem o mesmo resultado, retornando o valor inteiro 17.

A seguir, a conversão do exemplo mais complexo:

```
double Max(const DynamicArray<double> &A, int N)
{
    double X = A[0];

    for (int I = 1; I < N; I++)
    {
        if (X < A[I])
            X = A[I];
    }

    return X;
}
```

Observações

- O valor de retorno é definido explicitamente com return.
- A matriz dinâmica é representada por DynamicArray<double>.
- O parâmetro A é passado por referência constante para evitar cópias desnecessárias.
- A indexação da matriz dinâmica começa em 0.
- Variáveis locais (X e I) são declaradas no escopo da função.

Essas funções seguem o padrão normal de definição e retorno de valores em C++Builder.

Aqui está um exemplo mais complicado:

```
double Max(const DynamicArray<double> &A, int N)
{
    double X = A[0];

    for (int I = 1; I < N; I++)
    {
        if (X < A[I])
            X = A[I];
    }

    return X;
}
```

Nesse exemplo, a função percorre uma matriz dinâmica de valores reais e retorna o maior valor encontrado. O valor de retorno é definido explicitamente com a instrução return.

Outro exemplo mais elaborado:

```
double Power(double X, int Y)
{
    double Result = 1.0;
    int I = Y;

    while (I > 0)
    {
        if (I % 2 != 0)
            Result = Result * X;

        I = I / 2;
        X = X * X;
    }

    return Result;
}
```

Observações importantes

- Em C++Builder, o **valor de retorno** de uma função é definido por meio da instrução `return`.
- É possível calcular e atualizar o valor de retorno várias vezes dentro do corpo da função, desde que o valor final retornado seja compatível com o tipo declarado.
- Diferentemente de outras linguagens, o **nome da função não funciona como variável de retorno**.
- Para chamadas recursivas, o nome da função é usado normalmente como uma chamada de função.
- Uma variável local (como `Result`, neste exemplo) pode ser usada livremente em expressões, cálculos intermediários e operações internas antes de ser retornada.

Esse modelo torna o fluxo de retorno mais explícito e previsível, facilitando a leitura e a manutenção do código em C++Builder.

PARÂMETROS

Em C++Builder, **procedimentos e funções** normalmente recebem uma **lista de parâmetros**, que define os dados necessários para a execução da rotina.

Por exemplo, uma função com parâmetros é declarada assim:

```
double Power(double X, int Y);
```

Nesse caso, a **lista de parâmetros** é `(double X, int Y)`.

A lista de parâmetros é composta por uma sequência de **declarações de parâmetros**, separadas por vírgulas e colocadas entre parênteses. Cada parâmetro é declarado pelo **tipo**, seguido do **nome do parâmetro**.

A lista de parâmetros define:

- **A quantidade** de parâmetros que a rotina recebe

- **A ordem** em que devem ser passados
- **O tipo** de cada parâmetro

Essas informações devem ser respeitadas quando a rotina é chamada, garantindo compatibilidade de tipo e comportamento correto durante a execução.

Parâmetros por valor e por referência

Em C++Builder, os parâmetros de funções podem ser passados **por valor** ou **por referência**, definindo se a função pode ou não modificar o valor original passado pelo chamador.

Parâmetro passado por valor

Quando um parâmetro é passado por valor, a função recebe **uma cópia** do valor original. Alterações feitas dentro da função **não afetam** a variável usada na chamada.

```
int DoubleByValue(int X) // X é passado por valor
{
    X = X * 2;
    return X;
}
```

Parâmetro passado por referência

Quando um parâmetro é passado por referência, a função recebe **acesso direto** à variável original. Qualquer modificação feita dentro da função **altera o valor da variável passada**:

```
int DoubleByRef(int &X) // X é passado por referência
{
    X = X * 2;
    return X;
}
```

Diferença de comportamento

Ambas as funções retornam o mesmo resultado, mas apenas DoubleByRef pode modificar o valor da variável fornecida pelo chamador, pois o parâmetro é passado por referência.

Esse mecanismo torna possível criar funções que retornam valores e, ao mesmo tempo, modificam parâmetros recebidos, de forma clara e eficiente em C++Builder.

Parâmetros constantes (const)

Em **C++Builder**, um parâmetro **constante** é declarado com a palavra-chave **const**. Ele se comporta como uma **variável local somente leitura** dentro da função ou procedimento.

Parâmetros **const** são semelhantes aos parâmetros passados por referência, mas com duas diferenças importantes:

- **Não podem ser modificados** dentro da função.
- Permitem que o compilador **otimize o código**, especialmente para tipos estruturados e String.

- Evitam que o parâmetro seja passado inadvertidamente por referência para outra rotina que possa modificá-lo. Exemplo:

```
int CompareStr(const String &S1, const String &S2);
```

Explicação

- `const` → indica que o parâmetro não pode ser alterado.
- `String` → tipo de string do C++Builder (VCL).
- `&` → passagem por referência, evitando cópia desnecessária.

Dentro da função, qualquer tentativa de modificar S1 ou S2 resultará em erro de compilação, garantindo segurança e melhor desempenho.

Parâmetros padrões

Em **C++Builder**, é possível definir valores padrão para parâmetros diretamente no cabeçalho de procedimentos ou funções. Os valores padrão só podem ser usados em parâmetros passados **por valor** ou marcados como **const**, e devem ser expressões constantes compatíveis com o tipo do parâmetro.

Parâmetros com valor padrão devem aparecer **após** os parâmetros sem valor padrão.

Declaração equivalente em C++Builder:

```
void FillArray(const System::DynamicArray<int> &A, int Value = 0);
```

Exemplo de implementação:

```
void FillArray(const System::DynamicArray<int> &A, int Value)
{
    for (int i = 0; i < A.Length; ++i)
        A[i] = Value;
}
```

Uso:

```
FillArray(MyArray); // Value assume 0
FillArray(MyArray, 5); // Value assume 5
```

Observações

- `System::DynamicArray<int>` representa um array dinâmico de inteiros.
- O uso de `const &` evita cópias desnecessárias.
- O valor padrão deve ser definido apenas na declaração do método (normalmente no arquivo .h)

Em **C++Builder**, a regra é a mesma: **parâmetros com valor padrão devem ficar no final da lista de parâmetros**. Depois que você define um parâmetro com valor padrão, **todos os seguintes também precisam ter valor padrão**.

Declaração inválida em C++Builder:

```
void __fastcall MyProcedure(int I = 1, String S); // ERRO de sintaxe
```

Declaração válida – parâmetro padrão no final:

```
void __fastcall MyProcedure(int I, String S = "");
```

Declaração válida com múltiplos valores padrão:

```
void __fastcall MyProcedure(int I = 1, String S = "");
```

Exemplo completo .h:

```
void __fastcall MyProcedure(int I, String S = "teste");
```

Exemplo completo .cpp:

```
void __fastcall MyProcedure(int I, String S)
{
    // código
}
```